PROBLEM-SHEET-1

1. **Write a Node.js program using REPL terminal to:**
   - **Perform basic arithmetic operations (addition, subtraction, multiplication, division)**
   - **Create and manipulate JavaScript variables**
   - **Test the Node.js Console methods (console.log, console.error, console.warn, console.table)**
   - **Use the underscore (_) variable to store previous results**
   - **Save your REPL session to a file named 'session1.js'**

app.js

```
// Arithmetic Operations
10 + 5
20 - 7
4 * 6
20 / 4

// Variables
let a = 10
let b = 5
a + b
a - b
a * b
a / b

// Using underscore (_) to access last result
_
_ + 100

// Console methods
console.log("This is log message")
console.error("This is error message")
console.warn("This is warning message")

// console.table
let students = [
  { name: "Karan", marks: 90 },
  { name: "Amit", marks: 85 },
  { name: "Riya", marks: 88 }
]
console.table(students)
```

OUTPUT:

Arithmetic Operations

```
C:\Users\KARAN>node
Welcome to Node.js v22.17.0.
Type ".help" for more information.
> 10 + 5
15
> 20 - 7
13
> 4 * 6
24
> 20 / 4
5
```

Variables

```
> let a = 10
undefined
> let b = 5
undefined
> a + b
15
> a - b
5
> a * b
50
> a / b
2
```

using underscore (_)

```
> _ + 100
102
```

console methods

```
> console.log("this is log message")
this is log message
undefined
> console.error("this is error message")
this is error message
undefined
> console.warn("this is warning message")
this is warning message
undefined
```

Console table

```
> let students = [
... { name: "Karan" , marks: 90},
... { name: "Smit" , marks: 85},
... { name: "Pari" , marks: 80}
... ]
undefined
> console.table(students)
```

(index)	name	marks
0	'Karan'	90
1	'Smit'	85
2	'Pari'	80

```
undefined
```

2. Create a Node.js application that demonstrates asynchronous programming:

- **Write a function that simulates a time-consuming operation using setTimeout**
- **Create multiple asynchronous functions and observe their execution order**
- **Compare synchronous vs asynchronous code execution with practical examples**
- **Display the output showing the non-blocking behavior of Node.js**

```
// 1. Function that simulates time-consuming operation using setTimeout

function longTask(name, delay) {
  setTimeout(() => {
    console.log(`Task ${name} finished after ${delay}ms`);
  }, delay);
}

// 2. Multiple asynchronous functions

console.log("Start Async Demo");

longTask("A", 2000);
longTask("B", 1000);
longTask("C", 1500);

console.log("End Async Demo");

// 3. Compare Synchronous Execution

function syncTask(name) {
  console.log(`Task ${name} executed`);
}

console.log("\nStart Sync Demo");

syncTask("1");
syncTask("2");
syncTask("3");

console.log("End Sync Demo");
```

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\2nd> node .\script.js
Start Async Demo
End Async Demo

Start Sync Demo
Task 1 executed
Task 2 executed
Task 3 executed
End Sync Demo
Task B finished after 1000ms
Task C finished after 1500ms
Task A finished after 2000ms
```

3. Set up a Node.js development environment and create your first application:

- **Install Node.js on your system**
- **Create a file named 'app.js' that prints "Hello World from Node.js"**
- **Execute the file through Command Line Interface**
- **Create a package.json file using npm init**
- **Add a start script to run your application**

Step 1: use command: npm init -y

Step 2: create “app.js” file

app.js

```
// 2. Create a file that prints Hello World
console.log("Hello World from Node.js");
```

Step 3: edit packagelock.json

```
 "main": "app.js",
 "scripts": {
  "start" : "node app.js"
 },
```

Step 4: run file using node app.js or npm start

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\3rd\3.1> node app.js
hello world from Node.js!
```

4. Create a custom module named 'calculator.js' that:

- **Exports functions for add, subtract, multiply, and divide operations**
- **Includes a function to find the square root of a number**
- **Create a main file 'main.js' that imports and uses all these functions**
- **Test the module with various input values**
- **Handle division by zero error appropriately**

calculator.js

```
// 1. Export functions for add, subtract, multiply, divide

function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

function multiply(a, b) {
  return a * b;
}

// 5. Handle division by zero
function divide(a, b) {
  if (b === 0) {
    return "Error: Division by zero not allowed";
  }
  return a / b;
}

// 2. Function to find square root
function squareRoot(n) {
  return Math.sqrt(n);
}

// Export all functions
module.exports = {
  add,
  subtract,
  multiply,
  divide,
  squareRoot
};
```

main.js

```
// 3. Import and use all functions
const calc = require("./calculator");

// 4. Test module with various inputs

console.log("Add:", calc.add(10, 5));
console.log("Subtract:", calc.subtract(10, 5));
console.log("Multiply:", calc.multiply(10, 5));
console.log("Divide:", calc.divide(10, 5));
console.log("Divide by zero:", calc.divide(10, 0));
console.log("Square Root:", calc.squareRoot(25));
```

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\4th> node main.js
Add: 15
Subtract: 5
Multiply: 50
Divide: 2
Divide by zero: Error: Division by zero not allowed
Square Root: 5
```

5. Develop a Node.js application using NPM:

- **Initialize a new project with package.json**
- **Install the following packages locally: express, nodemon, dotenv**
- **Install chalk package globally**
- **Create a script in package.json to run your app with nodemon**
- **Demonstrate the use of at least one installed package in your code**

Step 1: use command: npm init -y

Step 2: use command: npm install express nodemon dotenv

Step 3: use command: npm install -g chalk

Step 4: update package.json

```
{
  "scripts": {
    "start": "nodemon app.js"
  }
}
```

Step 5: create app.js

app.js

```
// 5. Demonstrate use of installed packages

// Using express
const express = require("express");

// Using dotenv
require("dotenv").config();

const app = express();

// Basic route
app.get("/", (req, res) => {
  res.send("Express server is running");
});

// Using environment variable
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log("Server running on port " + PORT);
});
```

Step 6: create .env and add:

PORT=5000

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\5th> npm start

> 5th@1.0.0 start
> nodemon app.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
[dotenv@17.3.1] injecting env (1) from .env -- tip: 🔐 prevent building .env in docker: https://dotenvx.com/prebuild
Server running on port 5000
```

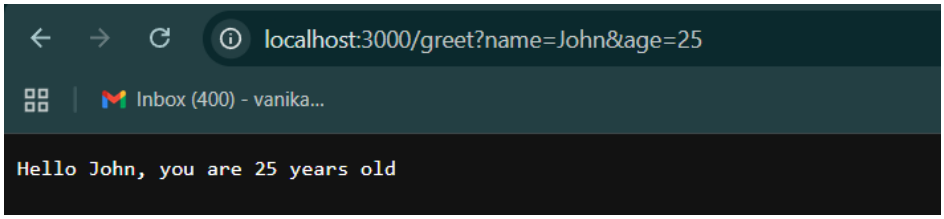
6. Write a program using built-in HTTP, URL, and Query String modules:

- **Create a server that listens on port 3000**
- **Accept URL: http://localhost:3000/greet?name=John&age=25**
- **Parse the URL and extract query parameters**
- **Display a message: "Hello John, you are 25 years old"**
- **Handle requests with missing parameters appropriately**

app.js

```
// 1. Import built-in modules
const http = require("http");
const url = require("url");
const querystring = require("querystring");

// 2. Create server listening on port 3000
const server = http.createServer((req, res) => {

  // 3. Parse URL and extract query parameters
  const parsedUrl = url.parse(req.url);
  const query = querystring.parse(parsedUrl.query);

  // Only handle /greet route
  if (parsedUrl.pathname === "/greet") {

    const name = query.name;
    const age = query.age;

    // 5. Handle missing parameters
    if (!name || !age) {
      res.writeHead(400, { "Content-Type": "text/plain" });
      res.end("Missing name or age parameter");
    } else {

      // 4. Display greeting message
      res.writeHead(200, { "Content-Type": "text/plain" });
      res.end(`Hello ${name}, you are ${age} years old`);
    }

  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Route not found");
  }
});

// Start server
server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

OUTPUT:



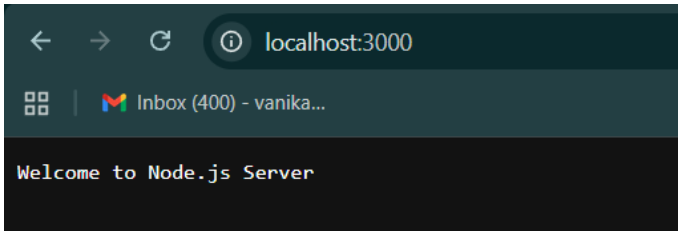
7. Create a web server that handles different types of HTTP requests:

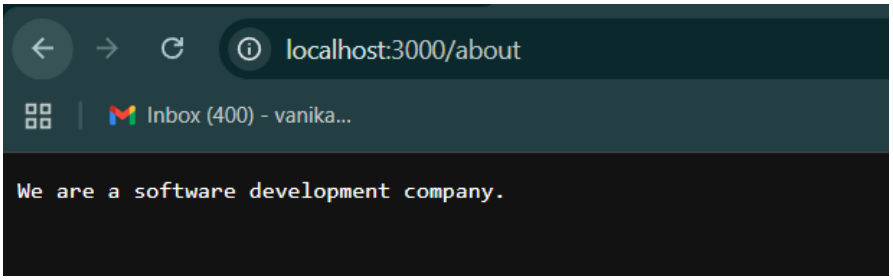
- **Handle GET request at '/' to display "Welcome to Node.js Server"**
- **Handle GET request at '/about' to display company information**
- **Handle POST request at '/submit' to receive form data**
- **Set appropriate status codes (200, 404, 500)**
- **Display proper error messages for invalid routes**

```
// 1. Import http module
const http = require("http");

// 2. Create server
const server = http.createServer((req, res) => {

  // Handle GET /
  if (req.method === "GET" && req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Welcome to Node.js Server");
  }

  // Handle GET /about
  else if (req.method === "GET" && req.url === "/about") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("We are a software development company.");
  }

  // Handle POST /submit
  else if (req.method === "POST" && req.url === "/submit") {

    let body = "";

    // 3. Receive form data
    req.on("data", chunk => {
      body += chunk.toString();
    });

    req.on("end", () => {
      res.writeHead(200, { "Content-Type": "text/plain" });
      res.end("Form Data Received: " + body);
    });
  }
```

```
// 5. Invalid Routes
  else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Error 404: Route Not Found");
  }

});

// Start server
server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

OUTPUT:






```
C:\Users\KARAN>curl -X POST http://localhost:3000/submit -d "name=Karan"
Form Data Received: name=Karan
C:\Users\KARAN>
```

8. Implement a File Management System using Node.js fs module:

- **Write a program to create a file 'students.txt' and write student data**
- **Read the file content synchronously and display it**
- **Read the file content asynchronously and display it**
- **Append new student data to the existing file**
- **Rename the file from 'students.txt' to 'students_data.txt'**
- **Delete the file after displaying its contents**

app.js:

```
// 1. Import fs module
const fs = require("fs");

// -------------------------------------
// 1. Create file and write student data
// -------------------------------------
fs.writeFileSync("students.txt", "John - 90\nRiya - 85\nAmit - 88");

// -------------------------------------
// 2. Read file synchronously
// -------------------------------------
const syncData = fs.readFileSync("students.txt", "utf8");
console.log("Synchronous Read:\n", syncData);

// -------------------------------------
// 3. Read file asynchronously
// -------------------------------------
fs.readFile("students.txt", "utf8", (err, data) => {
  if (err) {
    console.log("Error reading file");
    return;
  }
  console.log("Asynchronous Read:\n", data);

  // -------------------------------------
  // 4. Append new student data
  // -------------------------------------
  fs.appendFileSync("students.txt", "\nKaran - 92");

  // -------------------------------------
  // 5. Rename file
  // -------------------------------------
  fs.renameSync("students.txt", "students_data.txt");

  // -------------------------------------
  // Display content after rename
  // -------------------------------------
  const finalData = fs.readFileSync("students_data.txt", "utf8");
  console.log("Final File Content:\n", finalData);
```

```
  // ------------------------------------
  // 6. Delete file
  // ------------------------------------
  fs.unlinkSync("students_data.txt");
  console.log("File deleted successfully");
});
```

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\8th> node app.js
Synchronous Read:
 Karan - 90
Smit - 85
Pari - 88
Asynchronous Read:
 Karan - 90
Smit - 85
Pari - 88
Final File Content:
 Karan - 90
Smit - 85
Pari - 88
Karan - 92
File deleted successfully
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\8th>
```

9. Create a directory management application:

- **Create a directory named 'uploads'**
- **Create subdirectories: 'uploads/images', 'uploads/documents', 'uploads/videos'**
- **List all files and directories inside 'uploads'**
- **Create a file inside 'uploads/documents' directory**
- **Remove empty subdirectories**
- **Display appropriate success/error messages for each operation**

app.js

```
// 1. Import fs module
const fs = require("fs");

// ------------------------------------
// 1. Create directory 'uploads'
// ------------------------------------
if (!fs.existsSync("uploads")) {
  fs.mkdirSync("uploads");
  console.log("uploads folder created");
}

// ------------------------------------
// 2. Create subdirectories
```

```
// ------------------------------------
const subDirs = ["uploads/images", "uploads/documents", "uploads/videos"];

subDirs.forEach(dir => {
  if (!fs.existsSync(dir)) {
    fs.mkdirSync(dir);
    console.log(dir + " created");
  }
});

// ------------------------------------
// 3. List files and directories
// ------------------------------------
const items = fs.readdirSync("uploads");
console.log("Contents of uploads:", items);

// ------------------------------------
// 4. Create file in uploads/documents
// ------------------------------------
fs.writeFileSync("uploads/documents/info.txt", "Document file created");
console.log("File created inside documents folder");

// ------------------------------------
// 5. Remove empty subdirectories
// ------------------------------------
// ------------------------------------
// 5. Remove empty subdirectories only
// ------------------------------------
subDirs.forEach(dir => {
  if (fs.existsSync(dir)) {

    const files = fs.readdirSync(dir);

    if (files.length === 0) {
      fs.rmdirSync(dir);
      console.log(dir + " removed");
    } else {
      console.log(dir + " not empty, skipped");
    }

  }
});
```

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\9th> node app.js
uploads/images created
Contents of uploads: [ 'documents', 'images', 'videos' ]
File created inside documents folder
uploads/images removed
uploads/documents not empty, skipped
uploads/videos removed
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\9th>
```

10. Implement an Event-driven application using EventEmitter:

- **Create a custom event emitter class**
- **Define events: 'start', 'data', 'end', 'error'**
- **Create listeners for each event**
- **Emit events with appropriate data**
- **Demonstrate the 'once' method to handle an event only one time**
- **Implement error event handling**

app.js

```
// 1. Import EventEmitter
const EventEmitter = require("events");

// ------------------------------------
// 1. Create custom event emitter class
class MyEmitter extends EventEmitter {}

const emitter = new MyEmitter();

// ------------------------------------
// 3. Create listeners for each event

// start event
emitter.on("start", () => {
  console.log("Process Started");
});

// data event
emitter.on("data", (data) => {
  console.log("Data Received:", data);
});

// end event
emitter.on("end", () => {
  console.log("Process Ended");
});

// 6. Error event handling
emitter.on("error", (err) => {
  console.log("Error Occurred:", err.message);
});

// ------------------------------------
// 5. once() method (runs only once)
// ------------------------------------
emitter.once("onceEvent", () => {
  console.log("This runs only once");
});
```

```
// ------------------------------------
// 4. Emit events with data
emitter.emit("start");

emitter.emit("data", "Sample Data 1");
emitter.emit("data", "Sample Data 2");

emitter.emit("onceEvent");
emitter.emit("onceEvent"); // Will not run

emitter.emit("end");

// Emit error
emitter.emit("error", new Error("Something went wrong"));
```

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\10th> node .\app.js
Process Started
Data Received: Sample Data 1
Data Received: Sample Data 2
This runs only once
Process Ended
Error Occurred: Something went wrong
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\10th>
```

11. Write a Node.js application to connect with MySQL database:

- **Create a database named 'student_db'**
- **Create a table 'students' with fields: id, name, email, age**
- **Write code to insert 5 student records**
- **Retrieve and display all students**
- **Update a student's email address**
- **Delete a student record**
- **Display students in ascending order of names**

Install : npm install mysql2

app.js

```
// 1. Import mysql module
const mysql = require("mysql2");

// ------------------------------------
// Create MySQL Connection
// ------------------------------------
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "root",
  database: "student_db"
});
```

```
// Connect
db.connect(err => {
  if (err) throw err;
  console.log("Connected to MySQL");
});

// -------------------------------------
// 2. Create table students
// -------------------------------------
const createTable = `
CREATE TABLE IF NOT EXISTS students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50),
  email VARCHAR(50),
  age INT
)`;

db.query(createTable);

// -------------------------------------
// 3. Insert 5 student records
// -------------------------------------
const insertData = `
INSERT INTO students (name, email, age)
VALUES
('Karan', 'karan@gmail.com', 21),
('Amit', 'amit@gmail.com', 22),
('Riya', 'riya@gmail.com', 20),
('Neha', 'neha@gmail.com', 23),
('Rahul', 'rahul@gmail.com', 24)
`;

db.query(insertData);

// -------------------------------------
// 4. Retrieve and display all students
// -------------------------------------
db.query("SELECT * FROM students", (err, result) => {
  console.log("All Students:");
  console.table(result);
});

// -------------------------------------
// 5. Update student's email
// -------------------------------------
db.query(
  "UPDATE students SET email=? WHERE name=?",
  ["karan123@gmail.com", "Karan"]
);

// -------------------------------------
// 6. Delete a student record
```

```
// -------------------------------------
db.query("DELETE FROM students WHERE name='Rahul'");

// -------------------------------------
// 7. Display students in ascending order
// -------------------------------------
db.query(
  "SELECT * FROM students ORDER BY name ASC",
  (err, result) => {
    console.log("Students Sorted By Name:");
    console.table(result);
  }
);
```

use following commands in mysql shell:

```
CREATE DATABASE student_db;
USE student_db;
```

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\11th> node app.js
Connected to MySQL
All Students:
```

(index)	id	name	email	age
0	1	'Karan'	'karan@gmail.com'	21
1	2	'Amit'	'amit@gmail.com'	22
2	3	'Riya'	'riya@gmail.com'	20
3	4	'Neha'	'neha@gmail.com'	23
4	5	'Rahul'	'rahul@gmail.com'	24

```
Students Sorted By Name:
```

(index)	id	name	email	age
0	2	'Amit'	'amit@gmail.com'	22
1	1	'Karan'	'karan123@gmail.com'	21
2	4	'Neha'	'neha@gmail.com'	23
3	3	'Riya'	'riya@gmail.com'	20

12. Create an application demonstrating asynchronous event handling:

- **Create an EventEmitter that emits events asynchronously**
- **Pass multiple arguments (name, age, email) to event listeners**
- **Demonstrate how 'this' keyword works in event listeners**
- **Create a function that returns an EventEmitter**
- **Handle multiple events and show their execution sequence**

app.js

```
// 1. Import EventEmitter
const EventEmitter = require("events");

// ------------------------------------
// 4. Function that returns EventEmitter
// ------------------------------------
function createEmitter() {
  return new EventEmitter();
}

const emitter = createEmitter();

// ------------------------------------
// 2. Pass multiple arguments to listener
// ------------------------------------
emitter.on("user", function (name, age, email) {

  // 3. Demonstrate 'this'
  console.log("Event Name:", this.eventName);
  console.log("Name:", name);
  console.log("Age:", age);
  console.log("Email:", email);
});

// Another event
emitter.on("done", () => {
  console.log("All events completed");
});

// ------------------------------------
// 1. Emit events asynchronously
// ------------------------------------
console.log("Start");

setTimeout(() => {
  emitter.eventName = "user";
  emitter.emit("user", "Karan", 21, "karan@gmail.com");
}, 1000);

setTimeout(() => {
  emitter.emit("done");
```

```
}, 2000);

console.log("End");
```

OUTPUT:

```
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\12th> node app.js
Start
End
Event Name: user
Name: Karan
Age: 21
Email: karan@gmail.com
All events completed
PS C:\Users\KARAN\OneDrive\Documents\NODE JS PROBLEMSHEET QUESTIONS\12th>
```

13. Build a basic Express.js application:

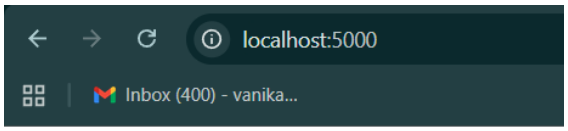
- **Install Express framework using NPM**
- **Create a server that runs on port 5000**
- **Define routes: '/', '/about', '/contact', '/products'**
- **Use route parameters to display product details: '/products/:id'**
- **Handle query strings: '/search?query=nodejs'**
- **Return JSON response for '/api/data' route**

app.js:

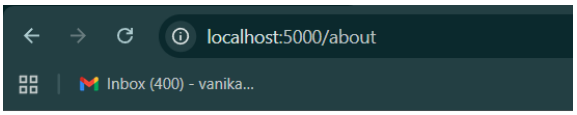
```
// 1. Import Express
const express = require("express");
const app = express();

// ------------------------------------
// 2. Create server on port 5000
// ------------------------------------
const PORT = 5000;

// ------------------------------------
// 3. Define basic routes
// ------------------------------------

// Home
app.get("/", (req, res) => {
  res.send("Welcome to Express App");
});

// About
app.get("/about", (req, res) => {
  res.send("This is About Page");
});

// Contact
app.get("/contact", (req, res) => {
```

```
  res.send("This is Contact Page");
});

// Products
app.get("/products", (req, res) => {
  res.send("Products Page");
});

// ------------------------------------
// 4. Route parameter
// ------------------------------------
app.get("/products/:id", (req, res) => {
  const id = req.params.id;
  res.send("Product ID: " + id);
});

// ------------------------------------
// 5. Handle query string
app.get("/search", (req, res) => {
  const query = req.query.query;
  res.send("Search Query: " + query);
});

// ------------------------------------
// 6. Return JSON response
app.get("/api/data", (req, res) => {
  res.json({
    name: "Karan",
    course: "NodeJS",
    status: "Active"
  });
});

// Start server
app.listen(PORT, () => {
  console.log("Server running on port " + PORT);
});
```

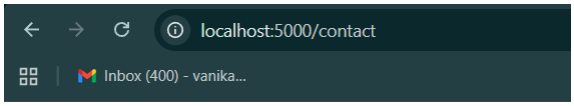
OUTPUT:



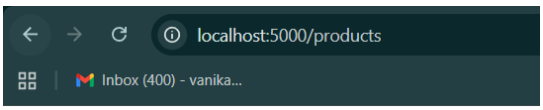
Welcome to Express App



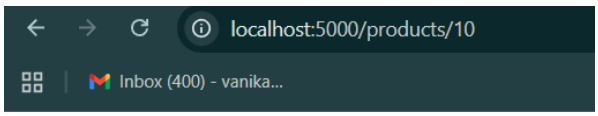
This is About Page



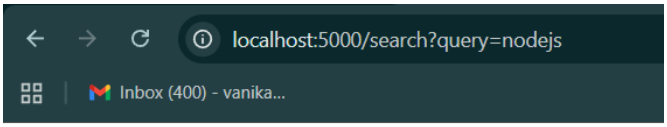
This is Contact Page



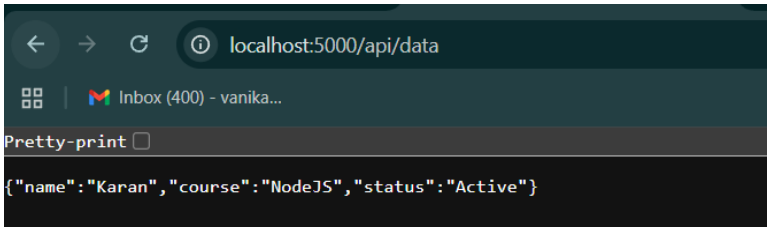
Products Page



Product ID: 10



Search Query: nodejs



14. Create an Express application for file upload:

- **Install and configure multer middleware**
- **Create an HTML form for file upload**
- **Accept single file upload (images only - jpg, png)**
- **Validate file size (max 2MB)**
- **Store uploaded files in 'uploads' directory**
- **Display success message with uploaded file details**
- **Implement cookie management to track number of uploads**

Install : npm install express multer cookie-parser

app.js

```
// 1. Import required modules
const express = require("express");
const multer = require("multer");
const path = require("path");
const cookieParser = require("cookie-parser");

const app = express();
app.use(cookieParser());

// ------------------------------------
// 1. Configure multer middleware
// ------------------------------------
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, "uploads/");
  },
  filename: function (req, file, cb) {
    cb(null, Date.now() + path.extname(file.originalname));
  }
});

// File filter (images only)
```

```
function fileFilter(req, file, cb) {
  const allowed = /jpg|jpeg|png/;
  const ext = allowed.test(path.extname(file.originalname).toLowerCase());
  if (ext) cb(null, true);
  else cb(new Error("Only JPG and PNG allowed"));
}

// Max size 2MB
const upload = multer({
  storage: storage,
  limits: { fileSize: 2 * 1024 * 1024 },
  fileFilter: fileFilter
});

// -------------------------------------
// 2. HTML Form
// -------------------------------------
app.get("/", (req, res) => {
  res.send(`
    <h2>Upload Image</h2>
    <form action="/upload" method="POST" enctype="multipart/form-data">
      <input type="file" name="image" required />
      <button type="submit">Upload</button>
    </form>
  `);
});

// -------------------------------------
// 3. Accept single file upload
// -------------------------------------
app.post("/upload", upload.single("image"), (req, res) => {

  if (!req.file) {
    return res.status(400).send("No file uploaded");
  }

  // 7. Cookie management (track uploads)
  let uploadCount = req.cookies.uploadCount || 0;
  uploadCount++;
  res.cookie("uploadCount", uploadCount);

  // 6. Success message with file details
  res.send(`
    <h3>File Uploaded Successfully</h3>
    <p>File Name: ${req.file.filename}</p>
    <p>Size: ${req.file.size} bytes</p>
    <p>Total Uploads: ${uploadCount}</p>
    <a href="/">Upload Again</a>
  `);
});

// Error handling
```

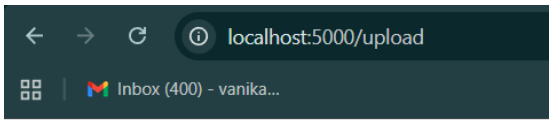
```
app.use((err, req, res, next) => {
  res.status(500).send(err.message);
});

// ------------------------------------
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

Create: mkdir uploads

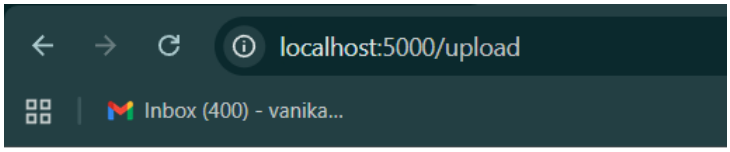
OUTPUT:

Upload Image

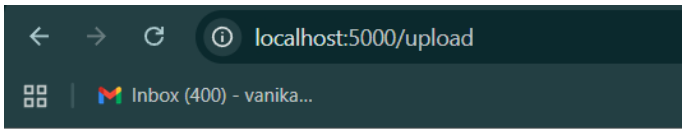
Choose File No file chosen Upload



Only JPG and PNG allowed



File too large



File Uploaded Successfully

File Name: 1771077522048.png

Size: 137350 bytes

Total Uploads: 1

Upload Again

15. Develop a complete Express application with template engine:

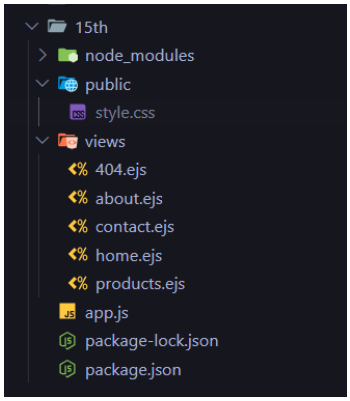
- **Install and configure EJS template engine**
- **Create views folder with the following templates:**

o home.ejs (homepage with navigation)
o about.ejs (about page)
o contact.ejs (contact form)
o products.ejs (display list of products)

- **Create routes to render each template**
- **Pass dynamic data to templates**
- **Serve static files (CSS, images) from 'public' folder**
- **Implement a 404 error page template**

Install : npm install express ejs

Folder Structure:



app.js

```
// 1. Import Express
const express = require("express");
const app = express();

// -------------------------------------
// 1. Configure EJS template engine
// -------------------------------------
app.set("view engine", "ejs");

// -------------------------------------
// 7. Serve static files
// -------------------------------------
app.use(express.static("public"));
app.use(express.urlencoded({ extended: true }));

// -------------------------------------
// Sample product data
// -------------------------------------
const products = [
  { id: 1, name: "Laptop", price: 50000 },
```

```
  { id: 2, name: "Mobile", price: 20000 },
  { id: 3, name: "Tablet", price: 15000 }
];

// ------------------------------------
// 4. Routes to render templates
// ------------------------------------
app.get("/", (req, res) => {
  res.render("home", { title: "Home Page" });   // 5. Dynamic data
});

app.get("/about", (req, res) => {
  res.render("about", { title: "About Us" });
});

app.get("/contact", (req, res) => {
  res.render("contact", { title: "Contact Us" });
});

app.get("/products", (req, res) => {
  res.render("products", { title: "Products", products });
});

// ------------------------------------
// 8. 404 Page
// ------------------------------------
app.use((req, res) => {
  res.status(404).render("404");
});

app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

home.ejs

```
<!DOCTYPE html>
<html>
<head>
  <title><%= title %></title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Home</h1>
  <nav>
    <a href="/">Home</a> |
    <a href="/about">About</a> |
    <a href="/contact">Contact</a> |
    <a href="/products">Products</a>
  </nav>
</body>
```

```
</html>
```

about.ejs

```
<h1><%= title %></h1>
<p>This is About Page</p>
<a href="/">Back</a>
```

contact.ejs

```
<h1><%= title %></h1>
<form>
  Name: <input type="text"><br><br>
  Email: <input type="email"><br><br>
  <button>Submit</button>
</form>
<a href="/">Back</a>
```

products.ejs

```
<h1><%= title %></h1>

<ul>
  <% products.forEach(p => { %>
    <li><%= p.name %> - ₹<%= p.price %></li>
  <% }) %>
</ul>
<a href="/">Back</a>
```

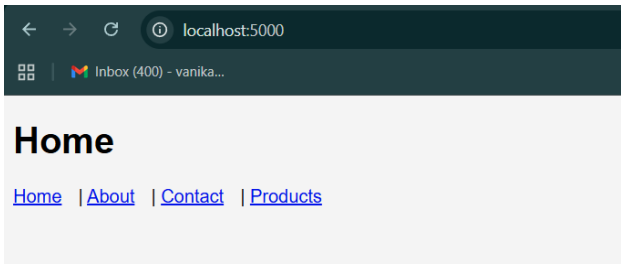
404.ejs

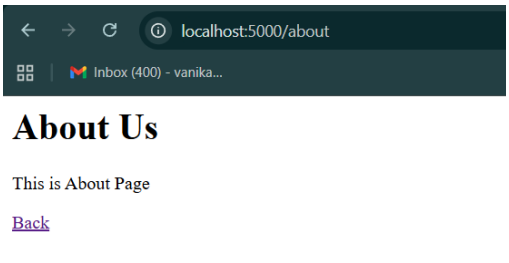
```
<h1>404 - Page Not Found</h1>
<a href="/">Go Home</a>
```

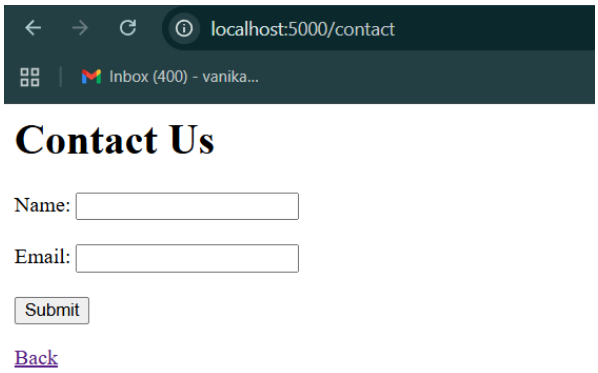
public/style.css

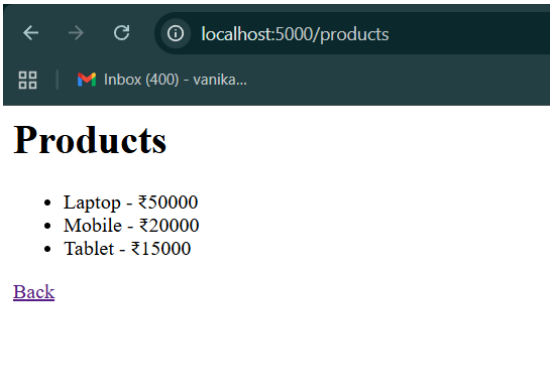
```
body {
  font-family: Arial;
  background: #f4f4f4;
}

nav a {
  margin-right: 10px;
}
```

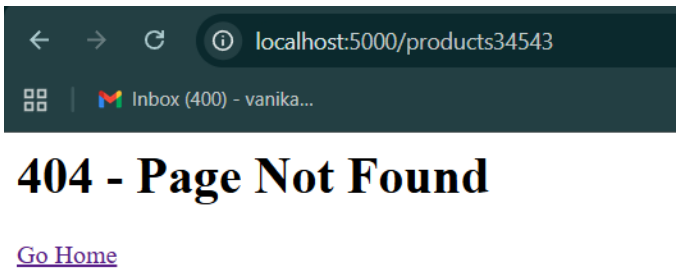
OUTPUT:


localhost:5000
Inbox (400) - vanika...
Home
Home | About | Contact | Products


localhost:5000/about
Inbox (400) - vanika...
About Us
This is About Page
Back


localhost:5000/contact
Inbox (400) - vanika...
Contact Us
Name:
Email:
Submit
Back


localhost:5000/products
Inbox (400) - vanika...
Products
Laptop - ₹50000
Mobile - ₹20000
Tablet - ₹15000
Back


localhost:5000/products34543
Inbox (400) - vanika...
404 - Page Not Found
Go Home